

Mapping performance cliffs in SAMtools sort: Memory, parallelism and temporary file I/O.

Benedicte April Jansen and Damian Matulis

Abstract—This project investigates how memory and parallelism affects the performance of samtools sort when sorting BAM files. We focus on interaction between memory per thread, number of threads, and temporary-file activity. We want to identify reproducible performance cliffs. The experiments vary the `-m` and `-@` parameters under controlled configurations, execution time, peak memory consumption, filesystem activity, and temporary-files will be measured using an automated python benchmark. We will repeat each configuration several times to account for variability, and select configuration that we also will test on a second BAM file for validation.

I. INTRODUCTION

High-throughput sequencing generates large amounts of sequence reads data that has to be processed before it can be used in genomic analyses. Genomic alignment sorting is, for those who are not familiar, the process of organizing DNA/RNA read alignments in a specific order, usually by their chromosomal coordinates on a reference-genome. The sequence alignment map (SAM) format and its compressed binary representation, binary alignment map (BAM), are commonly used formats to store sequencing reads aligned in a specific order, usually by their chromosomal coordinates on a reference genome [2]. SAMtools is a widely used suite of programs to process high-throughput sequence alignment data, and samtools sort provides a coordinate-based sorting [2].

Sorting large BAM-files places a large demand on both processing and storage resources. Since BAM files are compressed and can contain several gigabytes of alignment data, they must be decoded before they can be sorted and then encoded again when the sorted output is written. If the alignment data cannot be processed within the memory limit, samtools sort will create temporary files to the SSD, which later must be merged into the final BAM output. This additional storage can make BAM sorting both computationally and I/O intensive and can limit the performance benefits that are provided by additional threads [4].

A performance cliff is a sudden reduction in performance by a relatively small change in resources. In this study, the term performance cliff is used when describing a reproducible reduction in performance caused by a small change in resource configurations. During BAM sorting, a cliff may occur when reduced memory or increased resources causes an increase in temporary-file activity. Under a fixed memory budget,

increasing the thread count will reduce the amount of memory that is available per thread.

This project will focus on investigating how per-thread memory setting and thread count interact in samtools sort. There will be measured execution time, memory consumption and temporary file activity across controlled configurations, and want to identify when performance cliffs occur and determine if increased temporary storage activity can explain the cliff.

Motivation and goal

Genomic datasets can be huge and require sizable computational resources to be processed. By choosing resource setting poorly the analysis time will increase, storage traffic will increase, and the total computational cost will increase. The goal of this project is to characterize the interaction between memory, parallelism, and temporary storage during BAM sorting, and to give evidence that can support better resource configurations.

Research questions

1. How does per-thread memory affect execution time, peak memory consumption, and temporary file activity when the thread count is fixed?
2. How does thread count affect execution time and temporary file activity when the total planned sorting memory budget is held approximately constant?
3. At which per-thread memory and thread count does reproducible performance cliffs occur, and how do they correlate with changes in temporary file activity?

We hypothesize that reducing per-thread memory at a fixed thread will increase temporary file activity. Under a fixed memory budget, we expect temporary-file activity to be stable as the thread count changes, while execution time will initially decrease and then increase with increasing thread count.

II. RELATED WORKS

Li et al. [2] introduces the SAM format and the SAMtools software suit, which is widely used for working with high-throughput sequencing data. Danecek et al. [7] later describes the continued development of SAMtools and its use in genomic data-processing pipelines. These studies establish

SAMtools as an important tool for sequencing data, but they do not focus on how different memory and thread configurations affect the performance of samtools sort.

Weeks and Luecke [4] optimized SAMtools sorting for large BAM files using OpenMP task parallelism and memory optimization. They found a greater speed for in-memory sorting than for external out-of-core sorting, showing that storage activity can limit the performance benefits of parallelism. Their work therefore demonstrates that both memory availability and storage I/O can influence the sorting performance. However, our project does not modify the SAMtools implementation. Instead, we study the released version of samtools sort and systematically vary its existing memory and thread parameters to identify performance cliffs from there.

Other studies have developed other tools, such as elPrep by Herzeel et al. [1] and Sambamba by Tarasov et al. [3] to reduce processing time and add additional functionality. In contrast to other studies, this project does not modify SAMtools or compare or implement new tools. Instead, there will be used the released samtools sort implementation and systematically vary its memory and thread settings. The goal is to systematically identify performance cliffs and explain these through measured temp-I/O, especially when thread count is changed in a fixed memory-budget.

A closely related study by Siebelt [9] analyzed temporary-file management and the performance of samtools sort 1.19.2. The study showed that the available sorting memory can affect number of temporary files and that low-memory configurations can lead to additional merge operations. The memory scaling experiment was performed on a single sorting thread. Our project extends this by varying thread counts and per-thread memory, including comparisons under a fixed memory budget, using an unmodified samtools sort.

Another relevant study by Liu et al. [11] compared several tools, including SAMtools, using different variant-calling strategies. Their study focuses on how these tools perform when identifying genetic variants, but also focuses on the computational performance and look at differences between the implementations. This shows that software implementation and parallelism can affect the performance of genomic analysis pipelines. However, this is different from our project in that we do not focus on variant-calling. They do not investigate how memory and thread configurations affect the performance of samtools sort. Our project focuses on this narrower problem.

III. SYSTEM

The system received a fixed unsorted BAM file as input. As you can see in figure 1, SAMtools reads and decompresses the BAM records before placing them in sorting buffers in memory. The `-m` parameter controls approximately how much memory is available per sorting-thread, while the `-@` parameter controls the number of sorting and compressing threads. The

configured memory for storing BAM is approximately the multiplication of these two parameters. If the memory limit is hit before the end of the input file, then SAMtools will create temporary BAM files, which are merged and compressed again into after sorting. The performance of this therefore depends on the CPU processing, available memory, thread coordination and storage I/O. Our benchmark will be measuring the time, memory consumption, and temporary file activity for each configuration it tests.

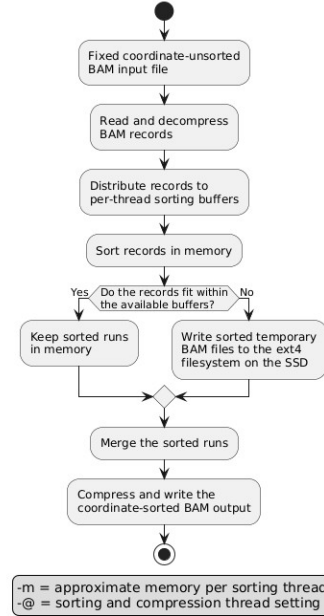


Fig 1. System model for sorting with samtools sort. Made using planttext.com.

IV. ENVIRONMENT AND SCOPE

When it comes to software and hardware, we are going to test everything on one controlled computer system. If time allows it, a small subset of configurations will be repeated on a second computer to examine if general performance patterns we found is hardware dependents. Results of these two systems would then be analyzed separately.

Dataset

The primary experiment will be using a BAM-file based on data from a study published by Ross-Inness et al. [8]. These files are already coordinate-sorted, and we will generate a non coordinate-sorted versions using samtools sort `-n`. This file will be generated once and then reused as fixed input through the experiments.

A second BAM-file will be used as a smaller validation experiment. Selected configurations from before, around and after the found performance cliff will be repeated with this file. This will allow us to observe if the identified performance pattern is specific to the primary BAM file.

Software environment

The software used is:

Host operating system: Windows 11

Execution environment: WSL version 2.7.12.0 with Ubuntu 24.04.2 LTS

Memory available for WSL2 to use: 7.5 GiB

Filesystem for experiments: ext4 with WSL2, stored on the SSD

Software: SAMtools 1.19.2 and HTSLib 1.19

Benchmark: implemented with SPython 3.12.3

Hardware environment

The hardware used is:

CPU: AMD Ryzen 9 5900HX with Radeon Graphics, 8 cores, 16 logical processors

Available memory: 15.4 GB RAM

Storage: INTEL SSDPEKNU010TZ, SSD, apx. 954 GB space

Workloads

The system's job for this project will be to use SAMtools sort on unsorted BAM-file in order to sort it with this program. The input used for that is a primary BAM-file first, and for validation of our experiments, a second BAM-file. The main parameters used in this project are the memory per thread parameter (-m) and number of threads parameter (-@). When it comes to the type of configurations tested, in the first experiment we will be testing fixed thread count and vary the -m. We will then be able to see how per-thread-memory affects execution time, memory consumption and temporary file-activity. For the second experiment, we will vary the thread count itself and hold the total memory around the same. This allows us to look at the interaction between memory and parallelism. In the third experiment, we will again hold the number of threads fixed, while we vary -m in a small range around the point where temporary files disappear. This is so we can look at the performance cliff in more detail. These workloads were chosen to give us an answer for the research questions, and also to explore memory, parallelism, and temporary file activity.

Evaluation and performance metrics

We are going to measure three different metrics. The first one is execution time. It will be measured from the start of samtools sort and stop when it's completed, and it will be reported in seconds. The next is peak memory consumption, which is going to check the maximum memory used that we can observe, and it will be checked for every sorting we run and report it in GiB. The last metric is temporary file activity, and it will be measured by using the temporary files we get from samtools sort when we execute the sorting. We will

record the number of temporary files created after each execution and the measurements from the executed runs with the same configuration will be compared to check on both the performance and the variation between each run.

Experimental protocols

All experiments use the same unsorted BAM file workload and the same hardware and software environment. Each configuration will be repeated five times. The order of the configuration is randomized, this is so that we reduce systematic effects from factors like changes in system temperature or background activity. For every run, samtools sort is executed with the selected -m and -@ values.

The main metric measured is wall-clock execution time, peak memory usage (RSS), number of temporary files, number of in-memory blocks, and filesystem I/O. The wall-clock time is measured externally by the benchmark script, while memory and filesystem statistics are collected using `usr/bin/time -v`. Temporary file information is collected from the samtools output. After each run, the resulting BAM file is checked using samtools quickcheck, the measurement written to a SV file, and the generated output and temporary files are removed before the next run.

Experiment 1 varies available memory per thread while thread count is fixed at 4. We will use the values [64M, 128M, 256M, 512M, 786M, 1024M]. Experiment 2 will vary the number of threads while keeping nominal memory budget constant at 3072 MiB. We will use configurations [-@ 1 -m 3072M, -@ 2 -m 1536M, -@ 4 -m 768M, -@ 8 -m 384M, and -@ 16 -m 192M]. Experiment 3 fixes the thread count at 4, while -m is varied in smaller steps around the point where temporary files disappear. The exact range has not yet been decided.

Possible confounders include background processes, CPU frequency changes, system temperature, filesystem and page-cache state, SSD activity, and other activity in the WSL environment. We cannot eliminate all these factors completely, but the effect is reduced by using the same machine and software configuration for all final runs, repeating every configuration five times, randomizing the run order, and avoiding other workloads during the experiments.

V. TIMELINE

We have a BAM-file and SAMtools installed, and have a first version of our benchmark. Next step will be testing it out properly with the first experiment and do necessary adjustments before we finalize the benchmark. When that is done, we can properly start doing our experiments. We will focus on the first experiments during the start, and move on to the second and third experiments later on. We will make sure to repeat the experiments on a second BAM-file because we want to see if the settings-optimality correspond to multiple BAM-files or even all of them, or would we need to change the settings for each BAM-file. Additionally, we want to run a subset of commands on another computer to see if there is a difference in performance on different computers.

VI. DISCUSSION

This project comes with several limitations that can affect our results. Firstly, all primary experiments will be executed on one hardware and software environment. The exact execution times and the location or performance cliff may be different on other CPUs, memory capacities, storage devices, operating systems, and SAMtools versions. This limits the generalizability of our results, but using one fixed system also gives a better controlled environment for comparing the tested memory and thread configurations. We will, as mentioned, if time allows, be conducting a second experiment by repeating a subset of commands on a second computer.

Secondly, the primary experiment will use one BAM-file. BAM files with different sizes or number of alignment records might have different sorting behavior. The project therefore does not aim to find one universally SAMtools configuration. Instead, it aims to characterize the interaction between per-thread memory, thread count, and temporary file activity for a controlled workload. To examine if this pattern is specific to one BAM file, we will also test these configurations using a second BAM file.

Finally, measurements can also be affected by variations caused by background processes, filesystem caching, etc. To reduce this risk, we use the same software and system configurations, unnecessary programs will be closed, and we will repeat each config several times.

The data for the measurements for each run will be stored in CSV-files. The CSV files need to have the configuration that is used, so each run needs to have correct configuration. When it comes to the raw data for the measurements, we will keep it in order to analyze it later on, but the temporary files and the output from the BAM-file will be removed after we validate it, in order to reduce the usage of storage. The results files will be organized so we can keep track of what result comes from what experiment.

REFERENCES

- [1] Herzeel, C., Costanza, P., Decap, D., Fostier, J., Reumers, J. (2015). elPrep: High-Performance Preparation of Sequence Alignment/Map Files for Variant Calling. *PLOS ONE* 10(7): e0132868. <https://doi.org/10.1371/journal.pone.0132868>
- [2] Li, H., Handsaker, B., Wysoker, A., Fennell, T., Ruan, J., Homer, N., Marth, G., Abecasis, G., Durbin, R. (2009). 1000 Genome Project Data Processing Subgroup, The Sequence Alignment/Map format and SAMtools, *Bioinformatics*, Volume 25, Issue 16, Pages 2078–2079, <https://doi.org/10.1093/bioinformatics/btp352>
- [3] Tarasov, A., Vilella, A. J., Cuppen E., Nijman, E. J., Prins, P. (2015). Sambamba: fast processing of NGS alignment formats, *Bioinformatics*, Volume 31, Issue 12, Pages 2032–2034, <https://doi.org/10.1093/bioinformatics/btv098>
- [4] Weeks, N.T., Luecke, G.R. (2017). Optimization of SAMtools sorting using OpenMP tasks. *Cluster Comput* 20, page 1869–1880. <https://doi.org/10.1007/s10586-017-0874-8>
- [5] Li, H. (2026, July 9). *Samtools sort – sorts SAM/BAM/CRAM files*. Samtools. <https://www.htslib.org/doc/samtools-sort.html>
- [6] Samtools. (n.d). *Samtools*. <https://www.htslib.org>
- [7] Danecek, P., Bonfield, J. K., Liddle, J., Marshall, J., Ohan, V., Pollard, M. O., Whitwham, A., Keane, T., McCarthy, S. A., Davies, R. M., & Li, H. (2021). Twelve years of SAMtools and BCFtools. *GigaScience*, 10(2), giab008. <https://doi.org/10.1093/gigascience/giab008>
- [8] Ross-Innes, C. S., Stark, R., Teschendorff, A. E., Holmes, K. A., Ali, H. R., Dunning, M. J., Brown, G. D., Gojis, O., Ellis, I. O., Green, A. R., Ali, S., Chin, S. F., Palmieri, C., Caldas, C., & Carroll, J. S. (2012). Differential oestrogen receptor binding is associated with clinical outcome in breast cancer. *Nature*, 481(7381), 389–393. <https://doi.org/10.1038/nature10730>
- [9] Siebelt, D. (2024). *Faster Sorting of Aligned DNA-Read Files* [Bachelor's thesis]. At the KIT Department of Informatics Institute of Theoretical Informatics (ITI).||
- [10] Vijaykumar, N., Hsieh, K., Pekhimenko, G., Khan, S., Shrestha, A., Ghose, S., Jog, A., Gibbons, P., Mutulu O. (2016). Zorua: A Holistic Approach to Resource Virtualization in GPUs. In *The 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO-49)*. IEEE Press, Article 15, 1–14. <https://dl.acm.org/doi/10.5555/3195638.3195656> ||
- [11] Liu, X., Han, S., Wang, Z., Gelernter, J., & Yang, B. Z. (2013). Variant callers for next-generation sequencing data: a comparison study. *PloS one*, 8(9), e75619. <https://doi.org/10.1371/journal.pone.0075619>